

Spring MVC and ORM

1. Overview of MVC

MVC (Model-View-Controller) is a software design pattern used to develop web applications by separating the application into three layers:

- **Model** – Handles data and business logic.
- **View** – Displays data to the user.
- **Controller** – Handles user requests and coordinates between Model and View.

Why do we need MVC?

Without MVC, all code (UI, business logic, and database operations) is written in a single class.

Problems:

- Difficult to maintain
- Difficult to debug
- Poor code reusability

2. Components of MVC

a. Control layer

The **Controller** receives HTTP requests from the user, processes them, calls the business logic, and returns a response or view.

Example

```
@Controller
public class StudentController {

    @GetMapping("/home")
    public String home() {
```

```
        return "home";  
    }  
}
```

b. **Model layer**

The model represents application data and business logic.

It usually contains:

1. POJO classes
2. Entity classes
3. Service layer
4. Repository layer

Example

```
public class Student {  
  
    private int id;  
    private String name;  
}
```

The `Student` object stores application data.

c. **View Layer**

The **View** is responsible for displaying information to the user.

Common View technologies:

- Thymeleaf
- JSP
- HTML
- FreeMarker

Example

```
<h1>Welcome Student</h1>
```

Spring sends this page to the browser.

3. Overview of ORM

What is ORM?

ORM is technique maps java objects to database tables.

Instead of writing SQL queries, we handle with java objects.

Without ORM

```
String sql = "SELECT * FROM student";
```

Manual SQL writing.

With ORM

```
Student student = repository.findById(1);
```

Spring Data JPA/Hibernate generates the SQL automatically.

4. Entity Example

```
@Entity
public class Student {

    @Id
    private int id;

    private String name;
}
```

Explanation

- `@Entity` → Represents a database table.
- `@Id` → Primary Key.

5. Querying

Spring Data JPA provides built-in methods for database operations.

Repository

```
public interface StudentRepository
    extends JpaRepository<Student,Integer> {
}
```

Save

```
repository.save(student);
```

Find All

```
repository.findAll();
```

Find By ID

```
repository.findById(1);
```

Delete

```
repository.deleteById(1);
```

No SQL is required.

7. Form Handlingconcept

Form Handling is the process of receiving user input through HTML forms.

HTML Form

```
<form action="/save" method="post">
  <input type="text" name="name">
  <button>Save</button>
</form>
```

Controller

```
@PostMapping("/save")
public String save(Student student) {

    return "success";
}
```

Spring automatically maps form fields to the `Student` object.

MVC vs ORM

MVC	ORM
Design Pattern	Mapping Technique
Separates application layers	Maps Java Objects to Database Tables
Handles request-response flow	Handles database operations
Uses Controller, Model, View	Uses Entity, Repository, Database